Dynamic structures:

Allocating memory to structure at runtime. It prevents memory wastage.

```
#include<stdio.h>

#include<conio.h>

#include<alloc.h> or #include<stdlib.h>

struct  stu

{

int id;

char name[20];

}s, *p;

void main()

{

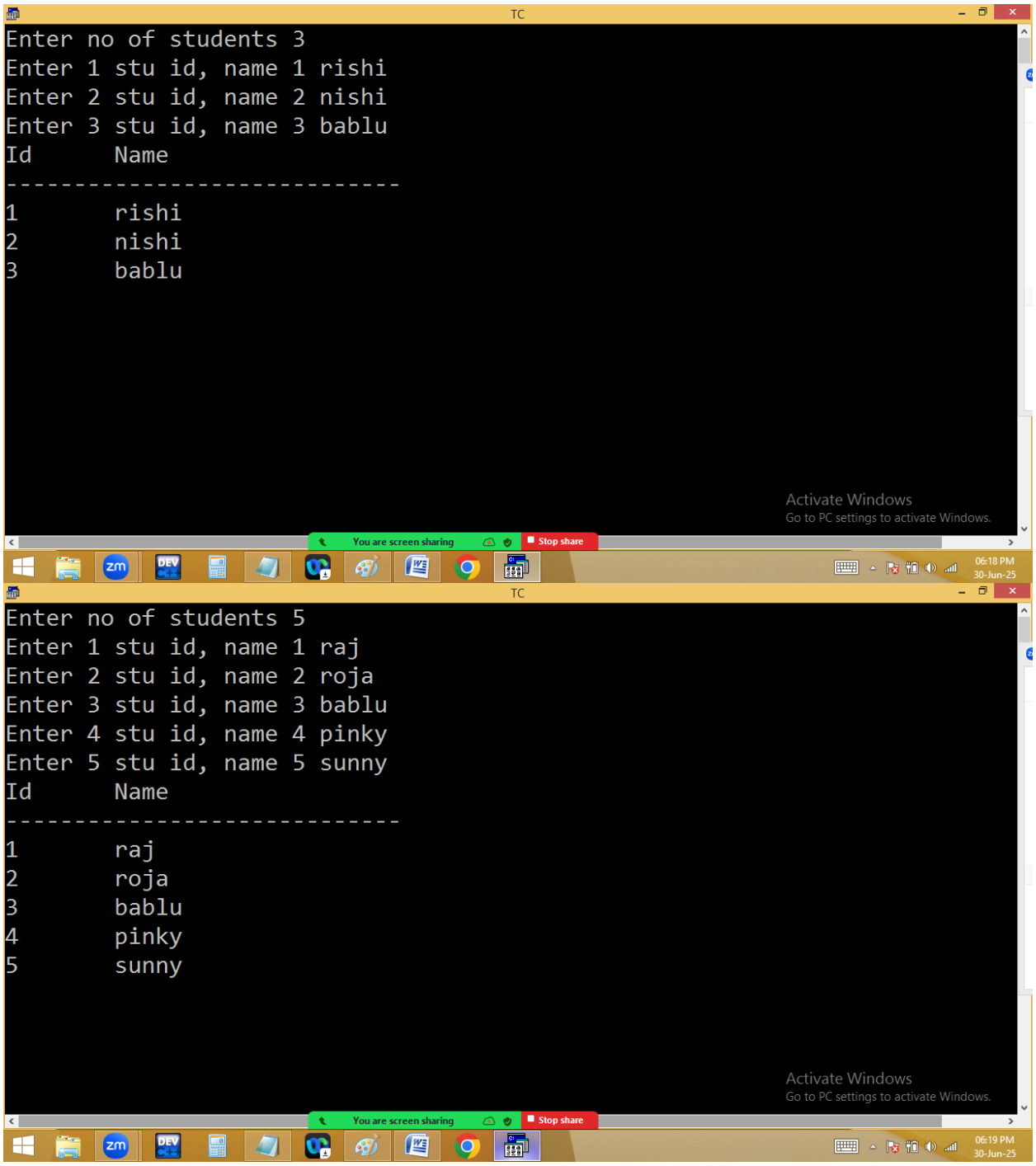
int n,i;

clrscr();

printf("Enter no of students ");scanf("%d",&n);

p = (struct stu *)calloc(n,sizeof(s));
```

```
for(i=0;i<n;i++)
{
printf("Enter %d stu id, name ", i+1);
scanf("%d %s",&p[i].id,p[i].name);
}
puts("Id\tName");
puts("----------------------------");
for(i=0;i<n;i++)printf("%d\t%s\n",p[i].id,p[i].name);
free(p); p=NULL; getch();
}
```



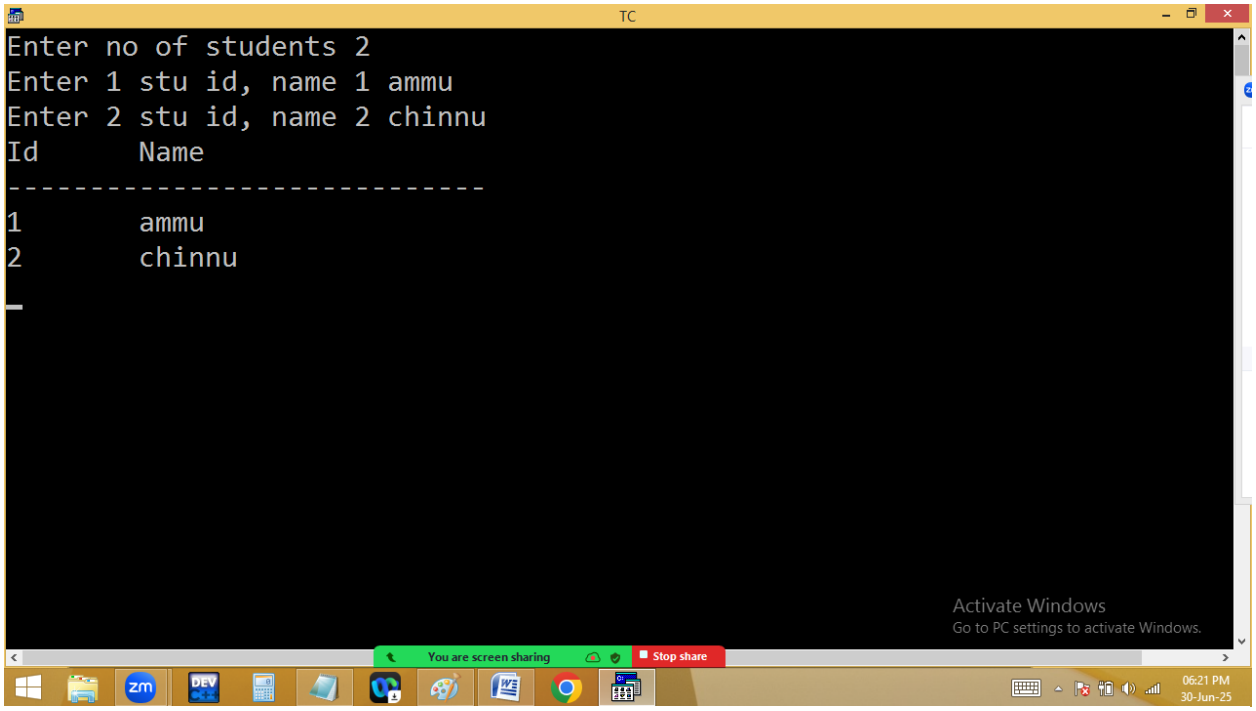

```
#include<stdio.h>

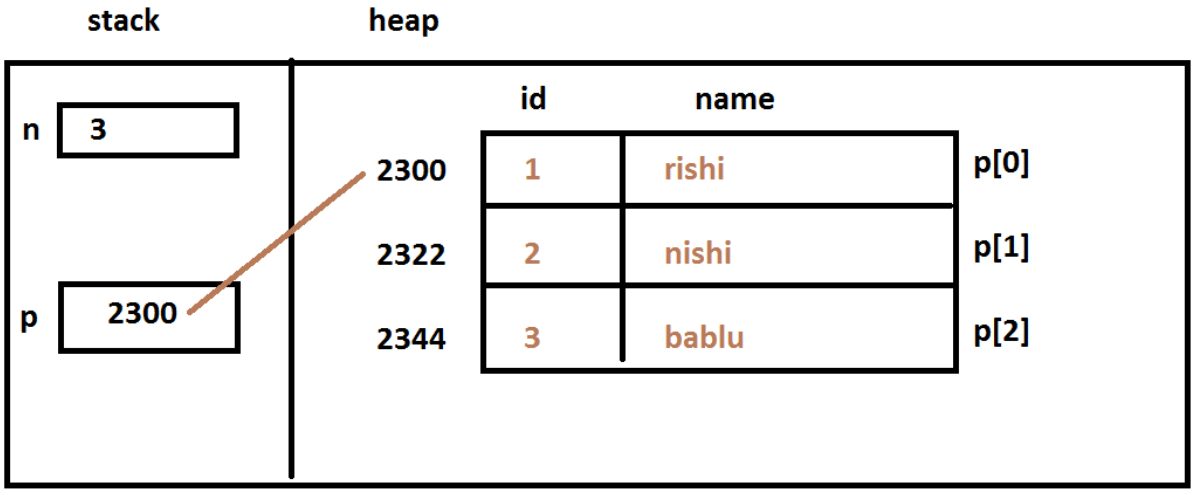
#include<conio.h>

struct  stu
```

```
{
int id;
char name[20];
}s, *p;
void main()
{
int n,i;
clrscr();
printf("Enter no of students ");scanf("%d",&n);
p = (struct stu *)calloc(n,sizeof(s));
for(i=0;i<n;i++)
{
printf("Enter %d stu id, name ", i+1);
scanf("%d %s",&(p+i)->id,(p+i)->name);
}
puts("Id\tName");
puts("--------------------------");
```

```
for(i=0;i<n;i++)printf("%d\t%s\n",(p+i)->id,(p+i)->name);
free(p); p=NULL; getch();
}
```





Nested / embedded structures: declaring one structure within another structure is called nested structure. It allows the concept of reusability and group of data.

```
#include<stdio.h>

#include<conio.h>

struct  bank

{

int acno;

char name[20];

};

struct loan

{

struct bank b; /* embedded structure */

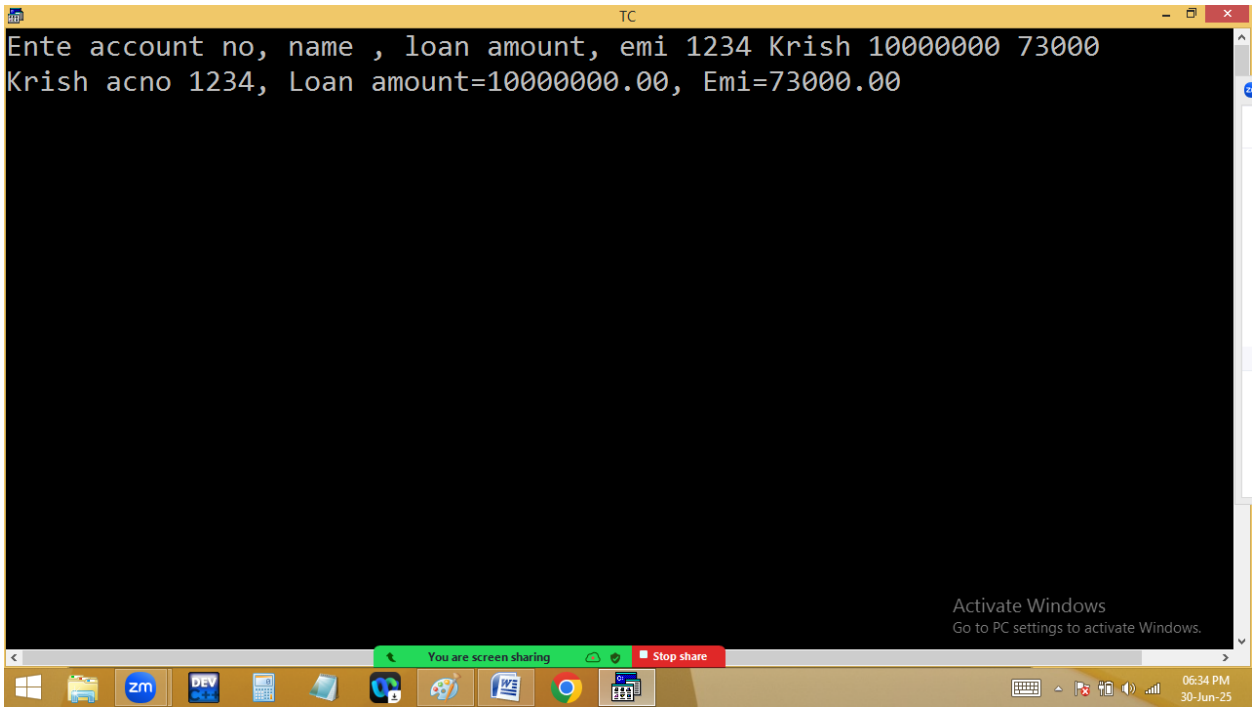
float amt,emi;

}L;

void main()

{
```

```
printf("Ente account no, name , loan amount, emi ");
scanf("%d %s %f %f",&L.b.acno,&L.b.name,&L.amt,&L.emi);
printf("%s acno %d, Loan amount=%.2f, Emi=%.2f",L.b.name,L.b.acno,L.amt,L.emi);
getch();
}
```

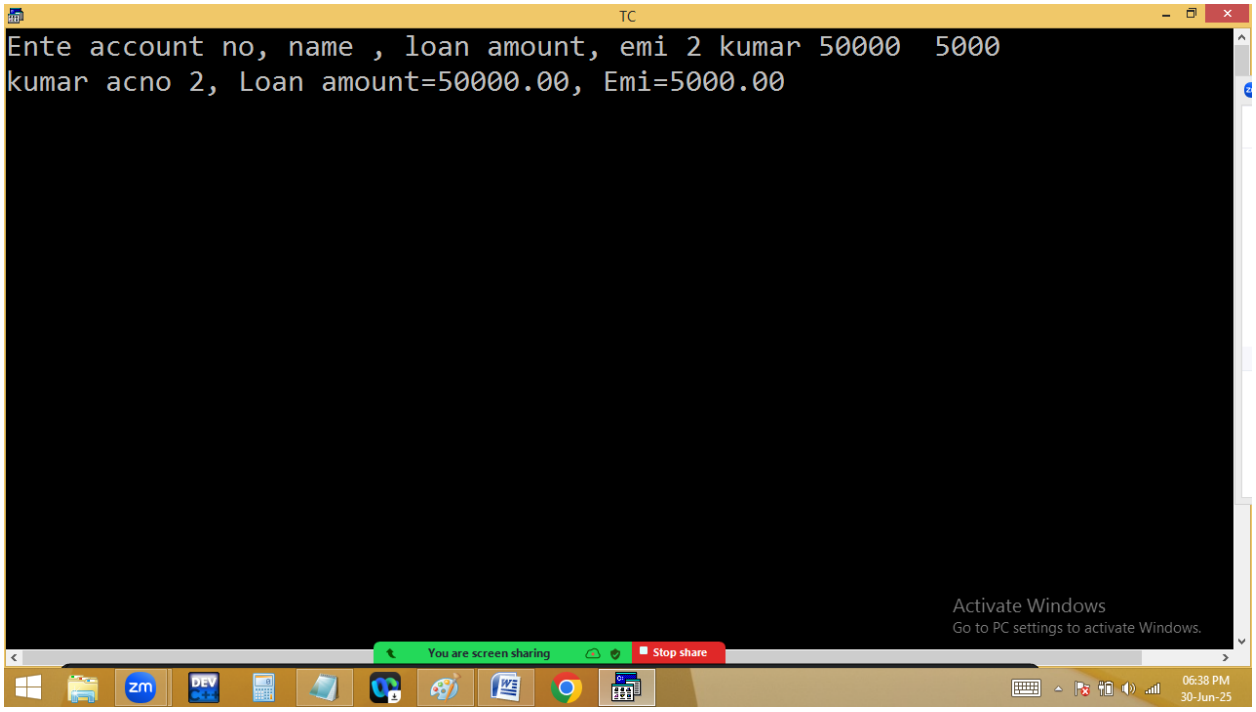



```
#include<stdio.h>
```

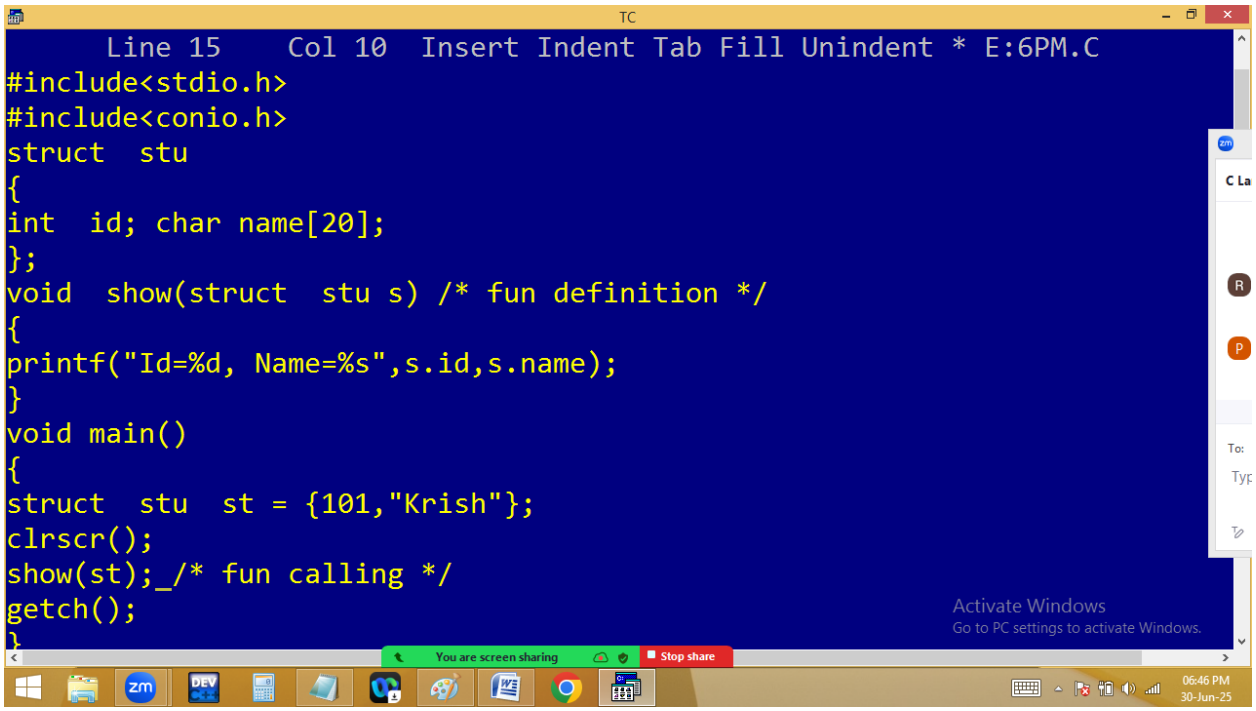
```
#include<conio.h>
struct  bank
{
int acno;
char name[20];
      struct loan  /* nested structure */
      {
      float amt,emi;
      }L;
}b;
void main()
{
printf("Ente account no, name , loan amount, emi ");
scanf("%d %s %f
%f",&b.acno,&b.name,&b.L.amt,&b.L.emi);
printf("%s acno %d, Loan amount=%.2f,
Emi=%.2f",b.name,b.acno,b.L.amt,b.L.emi);
```

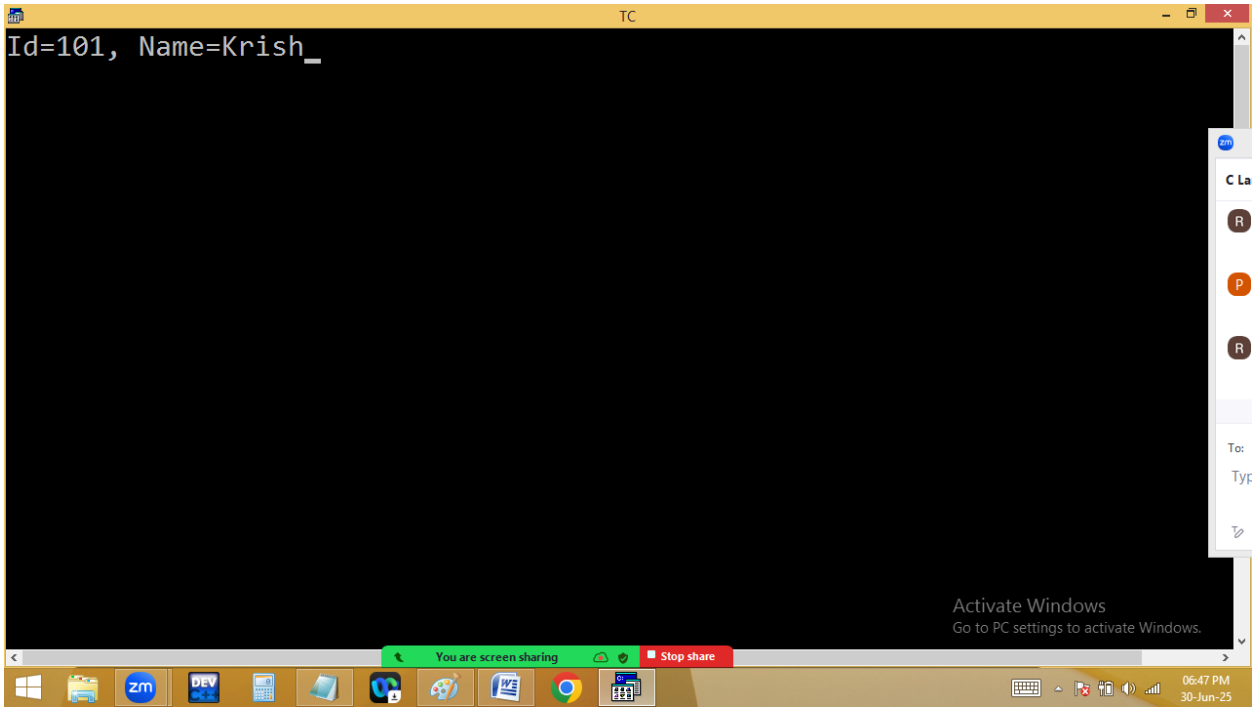
```
getch();

}
```



Passing structure to function:


TC
Line 15 Col 10 Insert Indent Tab Fill Unindent * E:6PM.C
#include<stdio.h>
#include<conio.h>
struct stu
{
int id; char name[20];
};
void show(struct stu s) /* fun definition */
{
printf("Id=%d, Name=%s",s.id,s.name);
}
void main()
{
struct stu st = {101,"Krish"};
clrscr();
show(st);_/* fun calling */
getch();
}
Activate Windows
Go to PC settings to activate Windows.
You are screen sharing
Stop share
06:46 PM
30-Jun-25

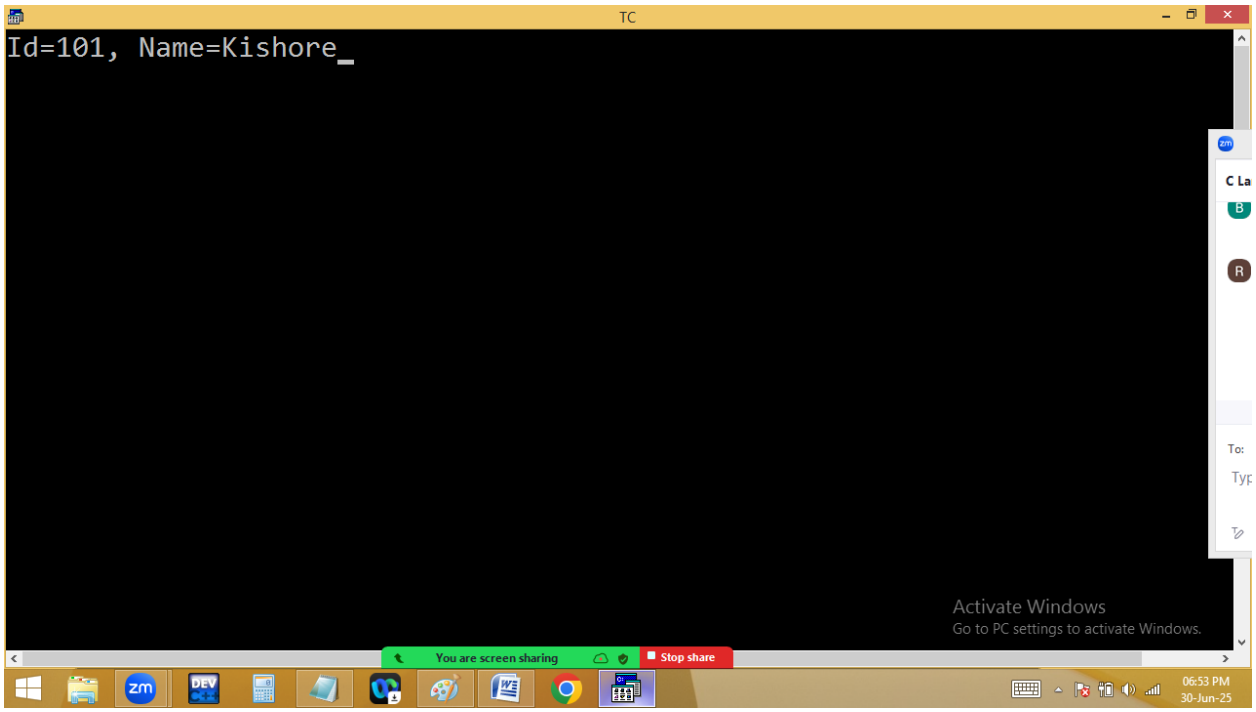

TC
Id=101, Name=Krish
Activate Windows
Go to PC settings to activate Windows.
You are screen sharing
Stop share
06:47 PM
30-Jun-25

Function returning structure:

```
#include<stdio.h>
#include<conio.h>
struct  stu
{
int  id; char name[20];
};
struct stu  show() /* fun definition */
{
struct stu s={101,"Kishore"};
return s;
}
void main()
{
struct  stu  st = show(); /* fun calling */
clrscr();
printf("Id=%d, Name=%s",st.id,st.name);
```

```
getch();

}
```



UNION

It is a user defined data type.

union is a keyword.

Like the structures unions are also used to place several variables of different data types under one name. But the main difference is all

union members are having common memory/ one memory.

In structure memory allocated for all the structure members separately. But in union one memory allocated and it is used by all the union members. We can access all the structure members at a time. But in union one member is active at a time. Structure size is sum of all the structure members. Union size is biggest union member size.

There is a situation where we have several options and user has to select one then go for union.

Syntax:

```
union    [ <union-tag-name> ]
{
datatype    variable1;
datatype    variable2;
} union variables;
```



➔Declaration and Accessing of union members is similar to the structures.

Eg: Finding union size:

```
#include<stdio.h>
#include<conio.h>
union   stu
{
int  id;
char name[20];
float fee;
}s;
void main()
{
printf("Union size  %d bytes", sizeof(s));
getch();
}
```

Output: Union size 20 bytes.

Eg:

Direct Initialization of union members:

Note: In union only the first member is initialized directly.

```
#include<stdio.h>
#include<conio.h>
union  stu
{
int  id;
char name[20];
float fee;
}s={1001,"krish",10000};
void main()
{
clrscr();
```

```
printf("Id=%d\nName=%s\nFee=%.2f",s.id,
s.name, s.fee);
getch();
}
```

Output: Initializer syntax error

Solution: s = {1001} ;

Eg:

Accessing union members:

```
#include<stdio.h>
#include<conio.h>
union  sample
{
int a;
long int b;
};
void main()
```

```
{
union   sample  s;
clrscr();
s.a=100;
printf("a = %d\n", s.a);
s.b=200;
printf("b = %ld", s.b);
getch();
}
```

Output

a = 100

b = 200

Differences between structure and union:

STRUCTURE	UNION
All the structure members are active at a time.	Only one union member is active at a time.
memory is allocated for	memory allocated for the

all the structure members	variable, which requires more memory in the union.
All the structure members initialized at a time.	Only the first member of union is initialized.
Structure size is sum of all the structure members.	Union size is the biggest variable data type size in the entire union.
They are useful to declare a compound data type to group data members related to person or item etc.	It is useful in certain situation where the user will select any one data member from group of data members.

typedef:

TYPEDEF

It is a keyword.

It is used to create new names for existing data types by providing **alias [another name]**.

The new name behavior is similar to the existing one and we can create any number of new variables from this alias name.

It is useful in structures to define the structure variables in other places.

typedef doesn't allows to create new data types with new features.

Syntax:

typedef datatype alias / newname;

newname variable, variable,….;

Eg:

```
#include<stdio.h>
#include<conio.h>

typedef int myint;

typedef float myfloat;

void main()

{
```

```
myint   a=100;
myfloat   b=1.2;
clrscr();
printf("a = %d\n", a);
printf("b = %.2f", b);
getch();
}
```

Using typedef in structures:

To declare structure variables in other places of the program, each time we should have to use struct keyword as follows.

```
struct   struct-tag-name   struvariables;
```

To avoid this problem declare the structure using typedef keyword.

Syntax:

```
typedef   struct
```

{

structure members;

}

alias ;

alias is the another name given for struct keyword. From this alias we can create the new structure variables as follows.

alias structure_variables;

eg:

```
#include<stdio.h>
#include<conio.h>

typedef  struct

{

int id;

char name[20];

}stu;   /* alias */
```

```
void main()
{
stu  s={101, “Krish”};
clrscr();
printf(“Id=%d,   Name = %s”, s.id, s.name);
getch();
}
```

Output:

Id =101

Name = Krish